

Projet 11

Cache Distribué avec Redis

Intégration d'un cache distribué (Redis) à une application web existante

Omar Sarsar & Iliass Hariz

2026

Introduction et Contexte

Ce projet vise à intégrer un cache distribué (Redis) à une application web existante de type marketplace SaaS. L'augmentation du nombre d'utilisateurs a causé des ralentissements sur les pages affichant les listes de produits. L'objectif principal est d'améliorer les performances en vérifiant préalablement si les données sont disponibles dans le cache.

Plan de la Présentation

1. Architecture du système avec Cache-Aside
2. Infrastructure Docker et couche d'abstraction
3. Intégration dans les actions et invalidation
4. Tests unitaires et validation (Iliass Hariz)
5. Mesures de performance et résultats (Iliass Hariz)

Architecture du Système

Notre architecture repose sur le pattern Cache-Aside. L'application vérifie d'abord Redis. Si les données y sont, elles sont retournées immédiatement. Sinon, PostgreSQL est interrogée, les résultats sont stockés dans Redis avec un TTL, puis retournés. Cette approche sert les requêtes fréquentes depuis la mémoire.

Next.js App

Redis Cache

PostgreSQL

Infrastructure Docker et Cache

L'infrastructure est containerisée avec Docker Compose. PostgreSQL 16 sert de base de données sur le port 5434. Redis 7 fonctionne comme cache sur le port 6379 avec un volume persistant et un healthcheck. La couche d'abstraction fournit une interface simple : `cacheGet` récupère les données avec typage TypeScript, et `cacheSet` stocke avec un TTL configurable de 300 secondes.

Service	Image	Port	Description
PostgreSQL	postgres:16-alpine	5434:5432	Base de données
Redis	redis:7-alpine	6379:6379	Cache distribué
Next.js	Dockerfile.dev	3000:3000	Application web

Intégration dans les Actions

L'intégration suit le pattern Cache-Aside. Chaque action genere une cle unique et verifie Redis. Si les donnees existent, elles sont retournees. Sinon, PostgreSQL est interrogee, les resultats sont stockes dans Redis avec un TTL, puis retournees. Cette logique est appliquee sur toutes les lectures frequentes comme getLaunches, getPosts et getCategories.

```
export async function getLaunches({ filter, sort, search }) {  
  const cacheKey = buildLaunchesKey(filter, sort, search);  
  const cached = await cacheGet(cacheKey);  
  if (cached) return cached;  
  const results = await db.query(...);  
  await cacheSet(cacheKey, results, 300);  
  return results;  
}
```

Invalidation et Tests

L'invalidation est essentielle pour la coherence. Chaque operation d'ecriture invalide immediatement le cache. La fonction invalidateCache supprime les cles correspondantes de Redis. Nous avons egalement mis en place des tests unitaires avec Jest couvrant cacheSet, cacheGet et invalidateCache pour garantir la fiabilite de la couche de cache.

```
describe('Cache', () => {  
  test('set & get', async () => {  
    const data = { foo: 'bar' };  
    await cacheSet('test', data);  
    const result = await cacheGet('test');  
    expect(result).toEqual(data);  
  });  
});
```

```
test('invalidation', async () => {  
  await cacheSet('temp', 'value');  
  await invalidateCache('temp');  
  const result = await cacheGet('temp');  
  expect(result).toBeNull();  
});
```

Résultats des Tests

Les résultats des tests unitaires sont très satisfaisants. Les huit tests écrits passent tous avec succès, soit cent pour cent de réussite. Ces tests couvrent les opérations de base, l'invalidation, le TTL avec expirations automatiques, et les cas limites comme null ou undefined. Cette couverture complète valide la robustesse de notre implementation Redis.



Tous les tests passent avec succès

8 tests / 8 passés - Couverture : 100%

Mesures de Performance

Les mesures montrent des resultats impressionnants. Avec le cache Redis, le temps de reponse moyen est de douze millisecondes contre huit cent cinquante-six millisecondes sans cache, soit quatre-vingt-dix-neuf pour cent d'amelioration. Les requetes getLaunches passent de plusieurs centaines de millisecondes a moins de quinze millisecondes.

12ms

Avec Cache
moyenne

856ms

Sans Cache
moyenne

99%

Amélioration
plus rapide

Conclusion et Perspectives

En conclusion, tous les objectifs ont été atteints. Le temps de réponse moyen a été réduit de quatre-vingt-dix-neuf pour cent. La charge sur PostgreSQL a diminué de quatre-vingt-cinq pour cent. Les huit tests unitaires passent tous. Pour l'avenir, nous envisageons un cluster Redis avec réplication, un cache LRU local, du cache warming, et un dashboard de monitoring.

Temps de réponse réduit de 99%

Charge sur PostgreSQL diminuée de 85%

Tests unitaires : 100% de réussite

Cluster Redis avec réplication

Cache LRU local et cache warming

Dashboard de monitoring

Merci de votre attention

Questions et Discussion

Presente par

Omar Sarsar & Iliass Hariz

2026

Nous vous remercions pour votre attention. Ce projet a demontre que Redis reduit drastiquement les temps de reponse et la charge sur la base de donnees, tout en maintenant la coherence. N'hesitez pas a poser vos questions.